

SRP en clases Java

vamos a contrastar una clase que utiliza SRP (principio de responsabilidad única) vs una que no, mostrando esta buena práctica en un metodo particular de dicha clase. Devolvamos el ejemplo en java

Principio de Responsabilidad Única — SRP

El SRP (Single Responsibility Principle) establece que:

Una clase debería tener una única responsabilidad y, por lo tanto, un único motivo para cambiar.

Vamos a trabajar con una clase `Alumno` y un método para registrar una nota.

1. Ejemplo que no aplica SRP

En este diseño, la clase `Alumno` realiza demasiadas tareas:

- Guarda los datos del alumno.
- Valida la nota.
- Calcula el promedio.
- Muestra información.
- Guarda información en una base de datos.
- Envía una notificación.

`</>` Java

```
import java.util.ArrayList;
import java.util.List;

public class Alumno {

    private String nombre;
    private List<Double> notas;

    public Alumno(String nombre) {
        this.nombre = nombre;
        this.notas = new ArrayList<>();
    }

    public void registrarNota(double nota) {

        // Responsabilidad 1: validar
        if (nota < 1 || nota > 10) {
```

```
        System.out.println("La nota debe estar entre 1 y 10.");
        return;
    }

    // Responsabilidad 2: modificar el estado del alumno
    notas.add(nota);

    // Responsabilidad 3: calcular el promedio
    double suma = 0;

    for (double notaActual : notas) {
        suma += notaActual;
    }

    double promedio = suma / notas.size();

    // Responsabilidad 4: mostrar información
    System.out.println("Nota registrada para: " + nombre);
    System.out.println("Promedio actual: " + promedio);

    // Responsabilidad 5: guardar en una base de datos
    System.out.println("Guardando la nota en la base de datos...");

    // Responsabilidad 6: enviar una notificación
    System.out.println("Enviando una notificación por correo...");
}
}
```

Problema del método registrarNota

El método:

Java

```
public void registrarNota(double nota)
```

no solamente registra una nota. También:

1. Valida.
2. Calcula.
3. Imprime.
4. Simula una persistencia.
5. Simula una notificación.

Por lo tanto, tiene **varios motivos para cambiar**.

Por ejemplo:

- Si cambia la forma de validar notas, hay que modificar `Alumno`.
- Si cambia la base de datos, hay que modificar `Alumno`.
- Si cambia la notificación, hay que modificar `Alumno`.

- Si cambia la presentación de los resultados, hay que modificar `Alumno`.

2. Ejemplo aplicando SRP

Ahora distribuimos las responsabilidades entre diferentes clases.

Clase `Alumno`

La responsabilidad de `Alumno` es representar al alumno y administrar sus notas.

```
</> Java
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Alumno {

    private String nombre;
    private List<Double> notas;

    public Alumno(String nombre) {
        this.nombre = nombre;
        this.notas = new ArrayList<>();
    }

    public void agregarNota(double nota) {
        notas.add(nota);
    }

    public String getNombre() {
        return nombre;
    }

    public List<Double> getNotas() {
        return Collections.unmodifiableList(notas);
    }
}
```

El método:

```
</> Java
public void agregarNota(double nota) {
    notas.add(nota);
}
```

tiene una única responsabilidad: **agregar una nota al alumno**.

Clase ValidadorNota

Su única responsabilidad es validar una nota.

```
</> Java
public class ValidadorNota {

    public boolean esValida(double nota) {
        return nota >= 1 && nota <= 10;
    }
}
```

Clase CalculadoraPromedio

Su única responsabilidad es calcular el promedio.

```
</> Java
import java.util.List;

public class CalculadoraPromedio {

    public double calcular(List<Double> notas) {

        if (notas.isEmpty()) {
            return 0;
        }

        double suma = 0;

        for (double nota : notas) {
            suma += nota;
        }

        return suma / notas.size();
    }
}
```

Clase AlumnoRepositorio

Su responsabilidad es guardar la información.

En este ejemplo solamente simulamos el acceso a la base de datos.

```
</> Java
public class AlumnoRepositorio {
```

```
public void guardar(Alumno alumno) {  
    System.out.println(  
        "Guardando los datos de " +  
        alumno.getNombre() +  
        " en la base de datos..."  
    );  
}  
}
```

Clase NotificadorAlumno

Su responsabilidad es enviar notificaciones.

</> Java

```
public class NotificadorAlumno {  
  
    public void notificarNotaRegistrada(Alumno alumno, double nota) {  
        System.out.println(  
            "Notificación: se registró la nota " +  
            nota +  
            " para el alumno " +  
            alumno.getNombre()  
        );  
    }  
}
```

Clase ServicioNotas

Esta clase coordina el caso de uso de registrar una nota.

</> Java

```
public class ServicioNotas {  
  
    private ValidadorNota validadorNota;  
    private AlumnoRepositorio alumnoRepositorio;  
    private NotificadorAlumno notificadorAlumno;  
  
    public ServicioNotas(  
        ValidadorNota validadorNota,  
        AlumnoRepositorio alumnoRepositorio,  
        NotificadorAlumno notificadorAlumno) {  
  
        this.validadorNota = validadorNota;  
        this.alumnoRepositorio = alumnoRepositorio;  
        this.notificadorAlumno = notificadorAlumno;  
    }  
}
```

```
public void registrarNota(Alumno alumno, double nota) {  
  
    if (!validadorNota.esValida(nota)) {  
        System.out.println("La nota debe estar entre 1 y 10.");  
        return;  
    }  
  
    alumno.agregarNota(nota);  
  
    alumnoRepositorio.guardar(alumno);  
  
    notificadorAlumno.notificarNotaRegistrada(alumno, nota);  
}  
}
```

Aunque `ServicioNotas` utiliza varias clases, su responsabilidad sigue siendo una sola:

Coordinar el caso de uso relacionado con el registro de una nota.

No realiza internamente la validación, la persistencia ni la notificación. Delega esas tareas.

3. Clase principal `App`

</> Java

```
public class App {  
  
    public static void main(String[] args) {  
  
        Alumno alumno = new Alumno("Lucía");  
  
        ValidadorNota validadorNota = new ValidadorNota();  
        AlumnoRepositorio alumnoRepositorio = new AlumnoRepositorio();  
        NotificadorAlumno notificadorAlumno = new NotificadorAlumno();  
  
        ServicioNotas servicioNotas = new ServicioNotas(  
            validadorNota,  
            alumnoRepositorio,  
            notificadorAlumno  
        );  
  
        servicioNotas.registrarNota(alumno, 8);  
        servicioNotas.registrarNota(alumno, 9);  
        servicioNotas.registrarNota(alumno, 15);  
  
        CalculadoraPromedio calculadoraPromedio =  
            new CalculadoraPromedio();  
  
        double promedio = calculadoraPromedio.calcular(  
            alumno.getNotas()  
        );  
    }  
}
```

```

    );

    System.out.println(
        "Promedio final de " +
        alumno.getNombre() +
        ": " +
        promedio
    );
}
}

```

Posible salida

Guardando los datos de Lucía en la base de datos...

Notificación: se registró la nota 8.0 para el alumno Lucía

Guardando los datos de Lucía en la base de datos...

Notificación: se registró la nota 9.0 para el alumno Lucía

La nota debe estar entre 1 y 10.

Promedio final de Lucía: 8.5

4. Estructura del proyecto

```

src
├── main
│   └── java
│       ├── Alumno.java
│       ├── ValidadorNota.java
│       ├── CalculadoraPromedio.java
│       ├── AlumnoRepositorio.java
│       ├── NotificadorAlumno.java
│       ├── ServicioNotas.java
│       └── App.java

```

5. Comparación

Sin SRP

Alumno valida la nota

Alumno calcula el
promedio

Aplicando SRP

ValidadorNota valida

CalculadoraPromedio
calcula

Sin SRP	Aplicando SRP
<code>Alumno</code> guarda en base de datos	<code>AlumnoRepositorio</code> guarda
<code>Alumno</code> envía notificaciones	<code>NotificadorAlumno</code> notifica
<code>Alumno</code> muestra resultados	<code>App</code> presenta el resultado
Un método realiza muchas tareas	Cada método tiene una tarea específica

Idea principal

El método problemático era:

```
</> Java
public void registrarNota(double nota)
```

porque mezclaba varias responsabilidades.

Al aplicar SRP, la operación se distribuye:

```
</> Java
validadorNota.esValida(nota);
alumno.agregarNota(nota);
alumnoRepositorio.guardar(alumno);
notificadorAlumno.notificarNotaRegistrada(alumno, nota);
```

Cada clase puede cambiar de manera independiente. Por ejemplo, podríamos reemplazar la notificación por consola por una notificación por correo sin modificar la clase `Alumno`.

